

```
In [2]: # Dependencies

# Standard Dependencies
import os
import numpy as np
import pandas as pd
from math import sqrt

# Visualization
from pylab import *
import matplotlib.mlab as mlab
import matplotlib.pyplot as plt
import seaborn as sns

# Statistics
from statistics import median
from scipy import signal
from scipy.special import factorial
import scipy.stats as stats
from scipy.stats import sem, binom, lognorm, poisson, bernoulli, spearmanr
from scipy.fftpack import fft, fftshift

# Scikit-Learn for Machine Learning models
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

```
In [4]: # Read in csv of Toy Dataset
# We will use this dataset throughout the tutorial
df = pd.read_csv(r"C:\Users\LIKHITH\OneDrive\Desktop\Data science\Data_files\toy_da
```

```
In [5]: df
```

Out[5]:

	Number	City	Gender	Age	Income	Illness
0	1	Dallas	Male	41	40367.0	No
1	2	Dallas	Male	54	45084.0	No
2	3	Dallas	Male	42	52483.0	No
3	4	Dallas	Male	40	40941.0	No
4	5	Dallas	Male	46	50289.0	No
...	...	...	...	...	...	...
149995	149996	Austin	Male	48	93669.0	No
149996	149997	Austin	Male	25	96748.0	No
149997	149998	Austin	Male	26	111885.0	No
149998	149999	Austin	Male	25	111878.0	No
149999	150000	Austin	Female	37	87251.0	No

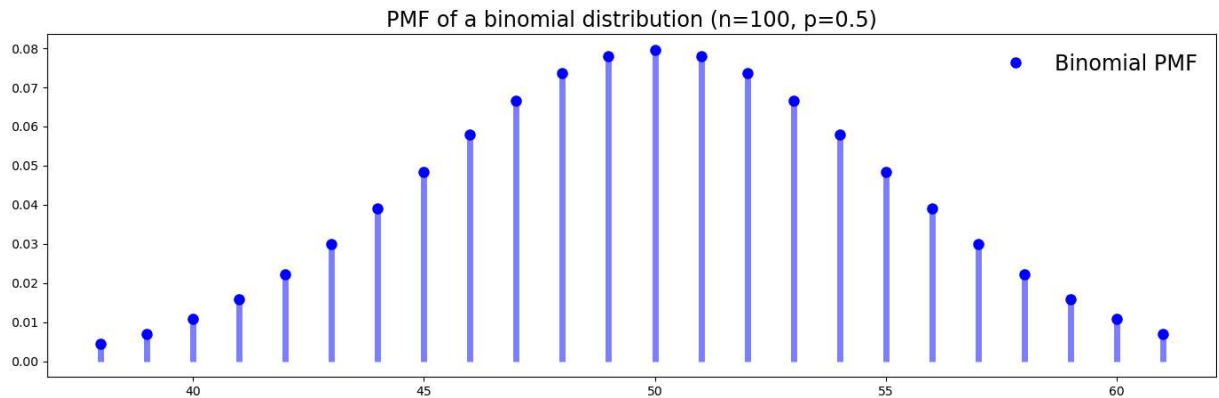
150000 rows × 6 columns

PMF (Probability Mass Function)

Here we visualize a PMF of a binomial distribution. You can see that the possible values are all integers

```
In [8]: # PMF Visualization
n = 100
p = 0.5

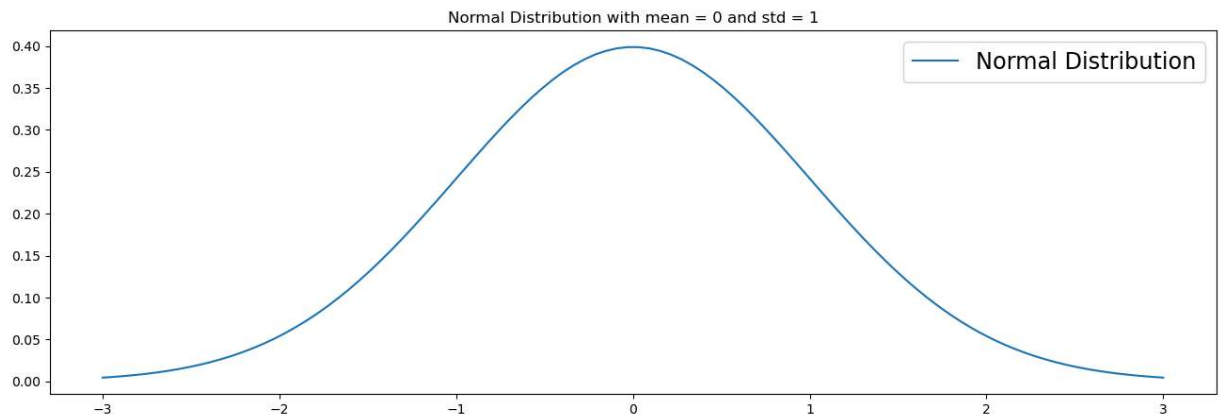
fig, ax = plt.subplots(1, 1, figsize=(17,5))
x = np.arange(binom.ppf(0.01, n, p), binom.ppf(0.99, n, p))
ax.plot(x, binom.pmf(x, n, p), 'bo', ms=8, label='Binomial PMF')
ax.vlines(x, 0, binom.pmf(x, n, p), colors='b', lw=5, alpha=0.5)
rv = binom(n, p)
#ax.vlines(x, 0, rv.pmf(x), colors='k', linestyle='-', lw=1, label='frozen PMF')
ax.legend(loc='best', frameon=False, fontsize='xx-large')
plt.title('PMF of a binomial distribution (n=100, p=0.5)', fontsize='xx-large')
plt.show()
```



PDF (Probability Density Functions)

The PDF is the same as a PMF, but continuous. It can be said that the distribution has an infinite number of possible values. Here we visualize a standard normal distribution with a mean of 0 and standard deviation of 1.

```
In [10]: # Plot normal distribution
mu = 0
variance = 1
sigma = sqrt(variance)
x = np.linspace(mu - 3*sigma, mu + 3*sigma, 100)
plt.figure(figsize=(16,5))
plt.plot(x, stats.norm.pdf(x, mu, sigma), label='Normal Distribution')
plt.title('Normal Distribution with mean = 0 and std = 1')
plt.legend(fontsize='xx-large')
plt.show()
```



CDF (Cumulative Distribution Function)

The CDF maps the probability that a random variable X will take a value of less than or equal to a value x ($P(X \leq x)$). CDF's can be discrete or continuous. In this section we visualize the continuous case. You can see in the plot that the CDF accumulates all probabilities and is therefore bounded between $0 \leq x \leq 1$.

```
In [11]: # Data
X = np.arange(-2, 2, 0.01)
```

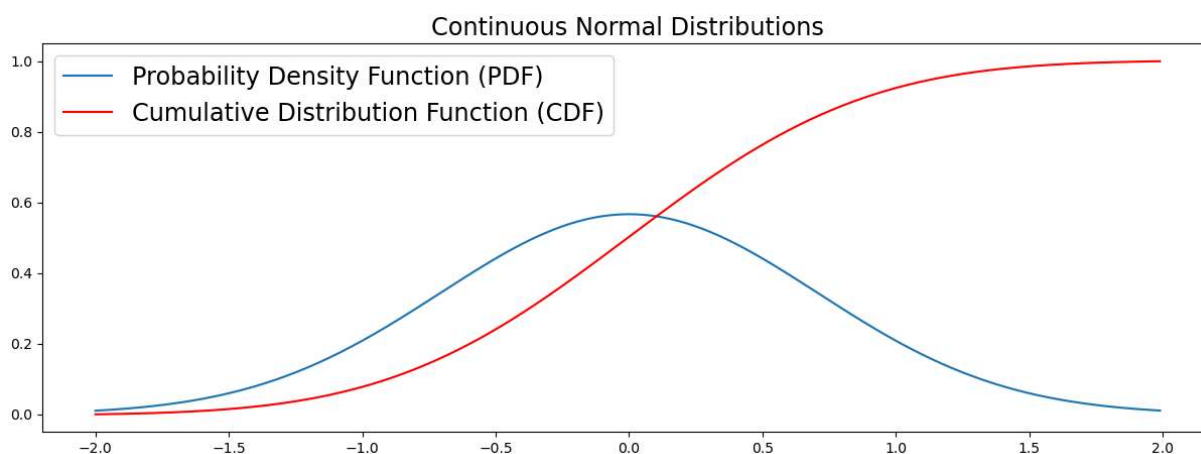
```

Y = exp(-X ** 2)

# Normalize data
Y = Y / (0.01 * Y).sum()

# Plot the PDF and CDF
plt.figure(figsize=(15,5))
plt.title('Continuous Normal Distributions', fontsize='xx-large')
plot(X, Y, label='Probability Density Function (PDF)')
plot(X, np.cumsum(Y * 0.01), 'r', label='Cumulative Distribution Function (CDF)')
plt.legend(fontsize='xx-large')
plt.show()

```



Distribution

Uniform Distribution

A Uniform distribution is pretty straightforward. Every value has an equal chance of occurring. Therefore, the distribution consists of random values with no patterns in them

```

In [12]: # Uniform distribution (between 0 and 1)
uniform_dist = np.random.random(1000)
uniform_df = pd.DataFrame({'value' : uniform_dist})
uniform_dist = pd.Series(uniform_dist)

```

```

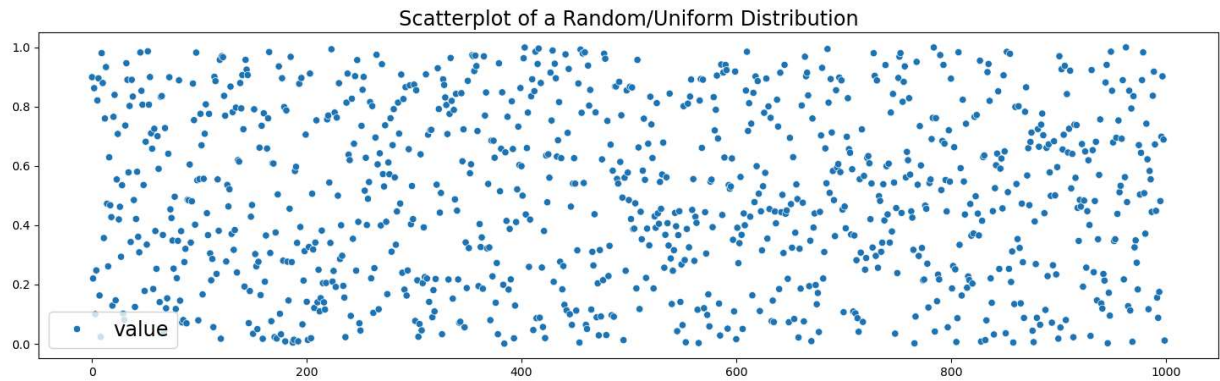
In [13]: plt.figure(figsize=(18,5))
sns.scatterplot(data=uniform_df)
plt.legend(fontsize='xx-large')
plt.title('Scatterplot of a Random/Uniform Distribution', fontsize='xx-large')

```

```

Out[13]: Text(0.5, 1.0, 'Scatterplot of a Random/Uniform Distribution')

```



```
In [14]: plt.figure(figsize=(18,5))
sns.distplot(uniform_df)
plt.title('Random/Uniform distribution', fontsize='xx-large')
```

C:\Users\LIKHITH\AppData\Local\Temp\ipykernel_6908\1098536719.py:2: UserWarning:

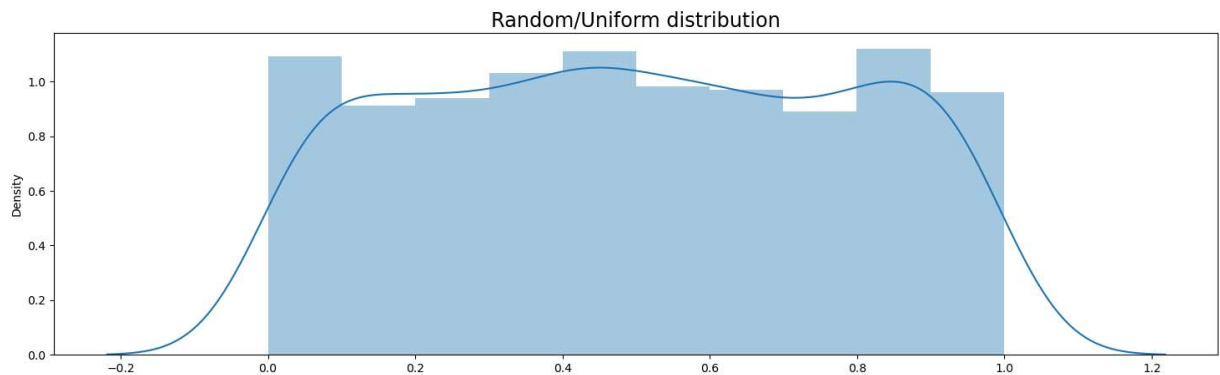
``distplot` is a deprecated function and will be removed in seaborn v0.14.0.`

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(uniform_df)
```

Out[14]: Text(0.5, 1.0, 'Random/Uniform distribution')



Normal Distribution

A normal distribution (also called Gaussian or Bell Curve) is very common and convenient. This is mainly because of the [Central Limit Theorem (CLT)] which states that with a large amount of independent random variables (like coin flips) the distribution tends towards a normal distribution.

```
In [15]: # Generate Normal Distribution
normal_dist = np.random.randn(10000)
normal_df = pd.DataFrame({'value' : normal_dist})
# Create a Pandas Series for easy sample function
```

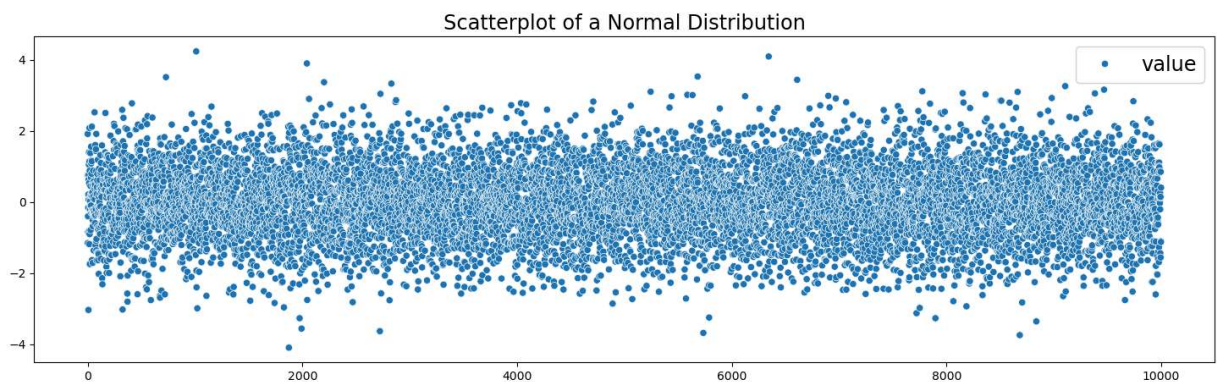
```
normal_dist = pd.Series(normal_dist)

normal_dist2 = np.random.randn(10000)
normal_df2 = pd.DataFrame({'value' : normal_dist2})
# Create a Pandas Series for easy sample function
normal_dist2 = pd.Series(normal_dist)

normal_df_total = pd.DataFrame({'value1' : normal_dist,
                                'value2' : normal_dist2})
```

```
In [16]: # Scatterplot
plt.figure(figsize=(18,5))
sns.scatterplot(data=normal_df)
plt.legend(fontsize='xx-large')
plt.title('Scatterplot of a Normal Distribution', fontsize='xx-large')
```

Out[16]: Text(0.5, 1.0, 'Scatterplot of a Normal Distribution')



```
In [17]: # Normal Distribution as a Bell Curve
plt.figure(figsize=(18,5))
sns.distplot(normal_df)
plt.title('Normal distribution (n=1000)', fontsize='xx-large')
```

C:\Users\LIKHITH\AppData\Local\Temp\ipykernel_6908\2492441267.py:3: UserWarning:

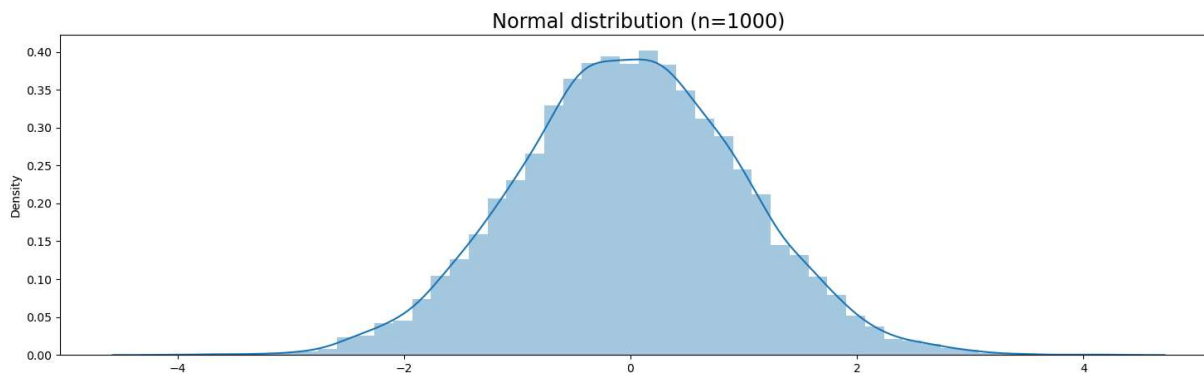
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(normal_df)
```

Out[17]: Text(0.5, 1.0, 'Normal distribution (n=1000)')



Binomial Distribution

A Binomial Distribution has a countable number of outcomes and is therefore discrete.

Binomial distributions must meet the following three criteria:

1. The number of observations or trials is fixed. In other words, you can only figure out the probability of something happening if you do it a certain number of times.
2. Each observation or trial is independent. In other words, none of your trials have an effect on the probability of the next trial.
3. The probability of success is exactly the same from one trial to another.

A **Bernoulli Distribution** is a special case of a Binomial Distribution.

All values in a Bernoulli Distribution are either 0 or 1.

```
In [18]: # Change of heads (outcome 1)
p = 0.6

# Create Bernoulli samples
bern_dist = bernoulli.rvs(p, size=1000)
bern_df = pd.DataFrame({'value' : bern_dist})
bern_values = bern_df['value'].value_counts()

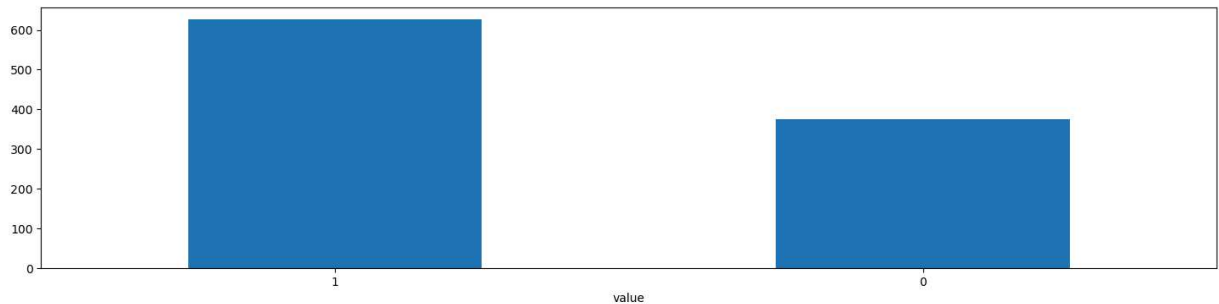
# Plot Distribution
plt.figure(figsize=(18,4))
bern_values.plot(kind='bar', rot=0)
plt.annotate(xy=(0.85,300),
             s='Samples that came up Tails\nn = {}'.format(bern_values[0]),
             fontsize='large',
             color='white')
plt.annotate(xy=(-0.2,300),
             s='Samples that came up Heads\nn = {}'.format(bern_values[1]),
             fontsize='large',
             color='white')
plt.title('Bernoulli Distribution: p = 0.6, n = 1000')
```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[18], line 12
    10 plt.figure(figsize=(18,4))
    11 bern_values.plot(kind='bar', rot=0)
----> 12 plt.annotate(xy=(0.85,300),
    13                 s='Samples that came up Tails\nn = {}'.format(bern_values[0]),
    14                 fontsize='large',
    15                 color='white')
    16 plt.annotate(xy=(-0.2,300),
    17                 s='Samples that came up Heads\nn = {}'.format(bern_values[1]),
    18                 fontsize='large',
    19                 color='white')
    20 plt.title('Bernoulli Distribution: p = 0.6, n = 1000')

```

TypeError: annotate() missing 1 required positional argument: 'text'



```
In [19]: bern_dist = bernoulli.rvs(p, size=1000)
```

Poisson Distribution

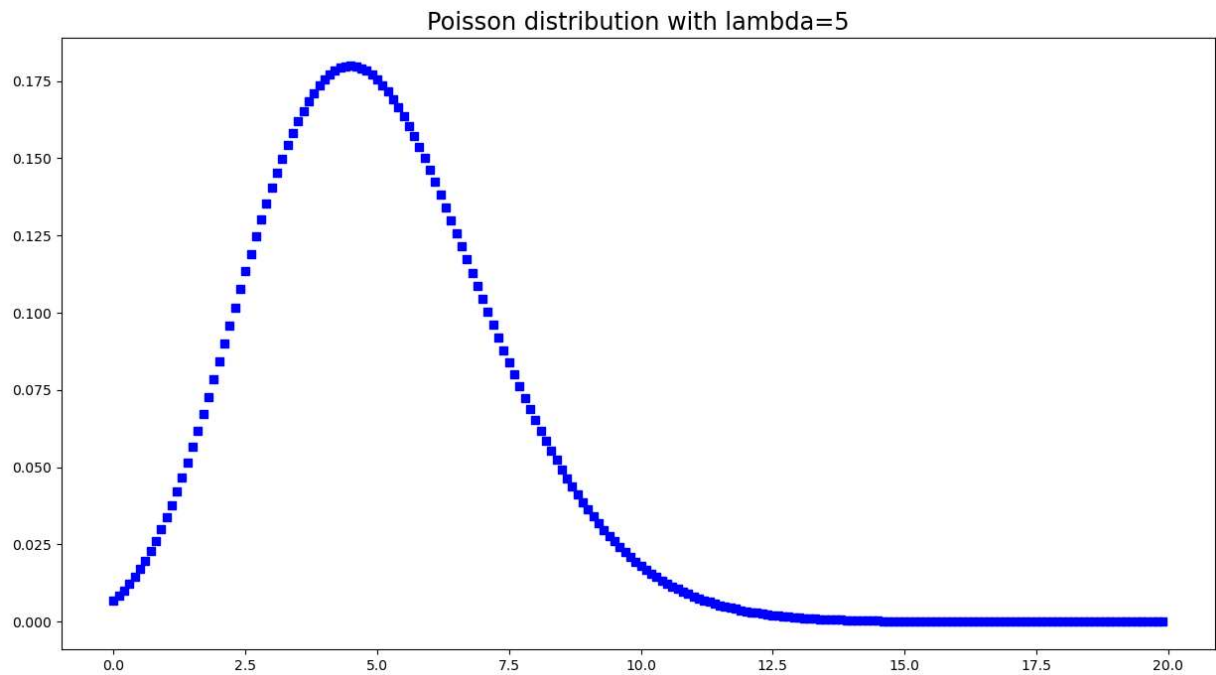
The Poisson distribution is a discrete distribution and is popular for modelling the number of times an event occurs in an interval of time or space.

```

In [20]: x = np.arange(0, 20, 0.1)
        y = np.exp(-5)*np.power(5, x)/factorial(x)

        plt.figure(figsize=(15,8))
        plt.title('Poisson distribution with lambda=5', fontsize='xx-large')
        plt.plot(x, y, 'bs')
        plt.show()

```



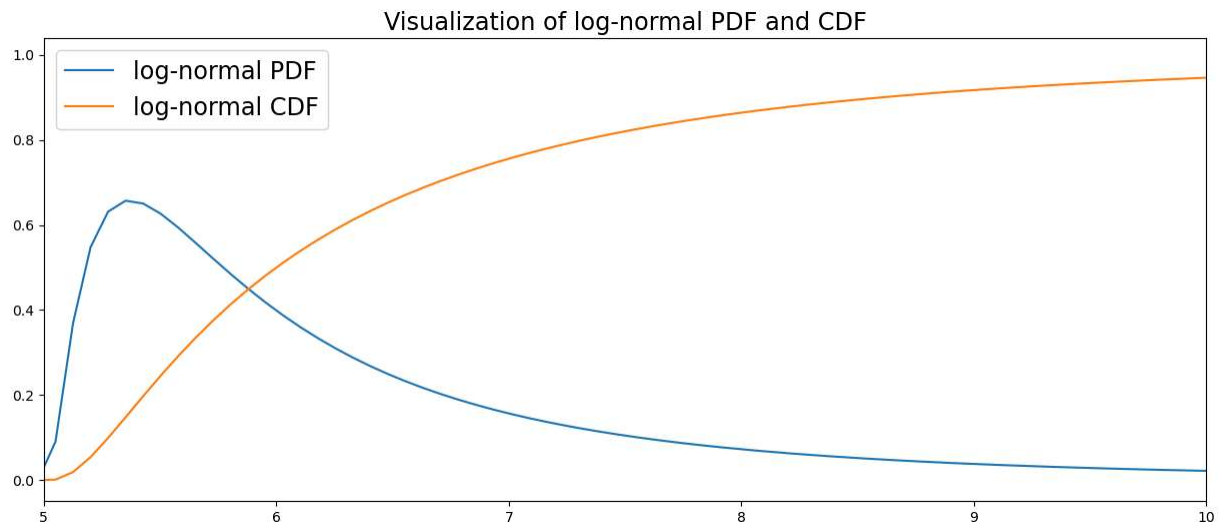
Log- Normal Distribution

A log-normal distribution is continuous. The main characteristic of a log-normal distribution is that its logarithm is normally distributed. It is also referred to as Galton's distribution.

```
In [21]: # Specify standard deviation and mean
std = 1
mean = 5

# Create Log-normal distribution
dist=lognorm(std,loc=mean)
x=np.linspace(0,15,200)

# Visualize log-normal distribution
plt.figure(figsize=(15,6))
plt.xlim(5, 10)
plt.plot(x,dist.pdf(x), label='log-normal PDF')
plt.plot(x,dist.cdf(x), label='log-normal CDF')
plt.legend(fontsize='xx-large')
plt.title('Visualization of log-normal PDF and CDF', fontsize='xx-large')
plt.show()
```



```
In [22]: # Summary
print('Summary Statistics for a normal distribution: ')
# Median
medi = median(normal_dist)
print('Median: ', medi)
display(normal_df.describe())

# Standard Deviation
std = sqrt(np.var(normal_dist))

print('The first four calculated moments of a normal distribution: ')
# Mean
mean = normal_dist.mean()
print('Mean: ', mean)

# Variance
var = np.var(normal_dist)
print('Variance: ', var)

# Return unbiased skew normalized by N-1
skew = normal_df['value'].skew()
print('Skewness: ', skew)

# Return unbiased kurtosis over requested axis using Fisher's definition of kurtosis
# (kurtosis of normal == 0.0) normalized by N-1
kurt = normal_df['value'].kurtosis()
print('Kurtosis: ', kurt)
```

Summary Statistics for a normal distribution:
 Median: -0.00871709112790214

	value
count	10000.000000
mean	-0.004919
std	1.000922
min	-4.094512
25%	-0.671484
50%	-0.008717
75%	0.664537
max	4.237051

The first four calculated moments of a normal distribution:

Mean: -0.0049193194548808475

Variance: 1.0017447633546175

Skewness: 0.02860880288800108

Kurtosis: 0.08450743191277743

```
In [23]: # Take sample
normal_df_sample = normal_df.sample(100)

# Calculate Expected Value (EV), population mean and bias
ev = normal_df_sample.mean()[0]
pop_mean = normal_df.mean()[0]
bias = ev - pop_mean
```

C:\Users\LIKHITH\AppData\Local\Temp\ipykernel_6908\2703849781.py:5: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```
ev = normal_df_sample.mean()[0]
```

C:\Users\LIKHITH\AppData\Local\Temp\ipykernel_6908\2703849781.py:6: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```
pop_mean = normal_df.mean()[0]
```

```
In [24]: print('Sample mean (Expected Value): ', ev)
print('Population mean: ', pop_mean)
print('Bias: ', bias)
```

Sample mean (Expected Value): -0.02755573783362948

Population mean: -0.0049193194548808475

Bias: -0.022636418378748635

```
In [25]: # Standard Error (SE)
uni_sample = uniform_dist.sample(100)
norm_sample = normal_dist.sample(100)

print('Standard Error of uniform sample: ', sem(uni_sample))
print('Standard Error of normal sample: ', sem(norm_sample))
```

```
# The random samples from the normal distribution should have a higher standard err
```

Standard Error of uniform sample: 0.031072761718799198

Standard Error of normal sample: 0.09861545581723706

Sampling methods

Non-Representative Sampling:

Convenience Sampling = Pick samples that are most convenient, like the top of a shelf or people that can be easily approached.

Haphazard Sampling = Pick samples without thinking about it. This often gives the illusion take you are picking out samples at random.

Purposive Sampling = Pick samples for a specific purpose. An example is to focus on extreme cases. This can be useful but is limited because it doesn't allow you to make statements about the whole population.

Representative Sampling:

Simple Random Sampling = Pick samples (psuedo)randomly.

Systematic Sampling = Pick samples with a fixed interval. For example every 10th sample (0, 10, 20, etc.).

Stratified Sampling = Pick the same amount of samples from different groups (strata) in the population.

Cluster Sampling = Divide the population into groups (clusters) and pick samples from those groups.

```
In [28]: # Note that we take very small samples just to illustrate the different sampling me

print('---Non-Representative samples:---\n')
# Convenience samples
con_samples = normal_dist[0:5]
print('Convenience samples:\n\n{}\n'.format(con_samples))

# Haphazard samples (Picking out some numbers)
hap_samples = [normal_dist[12], normal_dist[55], normal_dist[582], normal_dist[821]]
print('Haphazard samples:\n\n{}\n'.format(hap_samples))

# Purposive samples (Pick samples for a specific purpose)
# In this example we pick the 5 highest values in our distribution
purp_samples = normal_dist.nlargest(n=5)
print('Purposive samples:\n\n{}\n'.format(purp_samples))

print('---Representative samples:---\n')
```

```

# Simple (pseudo)random sample
rand_samples = normal_dist.sample(5)
print('Random samples:\n\n{}\n'.format(rand_samples))

# Systematic sample (Every 2000th value)
sys_samples = normal_dist[normal_dist.index % 2000 == 0]
print('Systematic samples:\n\n{}\n'.format(sys_samples))

# Stratified Sampling
# We will get 1 person from every city in the dataset
# We have 8 cities so that makes a total of 8 samples
df = pd.read_csv(r"C:\Users\LIKHITH\OneDrive\Desktop\Data science\Data_files\toy_da

strat_samples = []

for city in df['City'].unique():
    samp = df[df['City'] == city].sample(1)
    strat_samples.append(samp['Income'].item())

print('Stratified samples:\n\n{}\n'.format(strat_samples))

# Cluster Sampling
# Make random clusters of ten people (Here with replacement)
c1 = normal_dist.sample(10)
c2 = normal_dist.sample(10)
c3 = normal_dist.sample(10)
c4 = normal_dist.sample(10)
c5 = normal_dist.sample(10)

# Take sample from every cluster (with replacement)
clusters = [c1,c2,c3,c4,c5]
cluster_samples = []
for c in clusters:
    clus_samp = c.sample(1)
    cluster_samples.extend(clus_samp)
print('Cluster samples:\n\n{}\n'.format(cluster_samples))

```

---Non-Representative samples:---

Convenience samples:

```
0    1.915199
1   -1.153674
2    0.755161
3   -0.400337
4   -0.838110
dtype: float64
```

Haphazard samples:

```
[np.float64(1.3948966818434199), np.float64(-0.32348018789166044), np.float64(1.1242
56227163807), np.float64(0.5221058846859933), np.float64(-0.7295769039739903)]
```

Purposive samples:

```
1011    4.237051
6344    4.092709
2042    3.897606
5682    3.528864
729     3.512476
dtype: float64
```

---Representative samples:---

Random samples:

```
9160    0.043565
9762   -0.002751
998     0.035253
2809   -0.404874
7079   -1.176926
dtype: float64
```

Systematic samples:

```
0      1.915199
2000    0.413106
4000   -0.350243
6000   -0.887428
8000   -0.451357
dtype: float64
```

Stratified samples:

```
[46923.0, 102594.0, 82006.0, 122582.0, 88869.0, 59490.0, 116328.0, 90937.0]
```

Cluster samples:

```
[2.3033367073573934, 1.2542374476878952, 0.28376346294814436, 0.010395700791197532,
-0.45372762234203234]
```

Covariance

Covariance is a measure of how much two random variables vary together. variance is similar to covariance in that variance shows you how much one variable varies. Covariance tells you how two variables vary together.

```
In [27]: # Covariance between Age and Income
print('Covariance between Age and Income: ')

df[['Age', 'Income']].cov()
```

Covariance between Age and Income:

```
Out[27]:
```

	Age	Income
Age	133.922426	-3.811863e+02
Income	-381.186341	6.244752e+08

Correlation

Correlation is a standardized version of covariance. Here it becomes more clear that Age and Income do not have a strong correlation in our dataset.

```
In [29]: # Correlation between two normal distributions
# Using Pearson's correlation
print('Pearson: ')
print(df[['Age', 'Income']].corr(method='pearson'))

# Negatively correlated data
x1 = [1, 3, 3, 4, 6, 7, 7, 9, 10, 10]
x2 = [9.7, 9.3, 7, 6, 7, 7.2, 6, 4.2, 4.5, 4]

# Positively correlated data
y1 = [0, 1, 2, 2.3, 3, 4, 4, 7, 9, 9.5]
y2 = [2.2, 4, 3, 5, 5.3, 5, 7, 9, 5, 10]

# Correlation with NumPy
print('\nx1,x2 correlation: \n', np.corrcoef(x1, x2))
print('\ny1,y2 correlation: \n', np.corrcoef(y1, y2))
```

Pearson:

	Age	Income
Age	1.000000	-0.001318
Income	-0.001318	1.000000

x1,x2 correlation:

```
[[ 1.          -0.88237093]
 [-0.88237093  1.          ]]
```

y1,y2 correlation:

```
[[1.          0.7737071]
 [0.7737071  1.          ]]
```

```
In [30]: # Using Spearman's rho correlation
print('Spearman: ')
df[['Age', 'Income']].corr(method='spearman')
```

Spearman:

```
Out[30]:
```

	Age	Income
Age	1.000000	-0.001452
Income	-0.001452	1.000000

Linear Regression

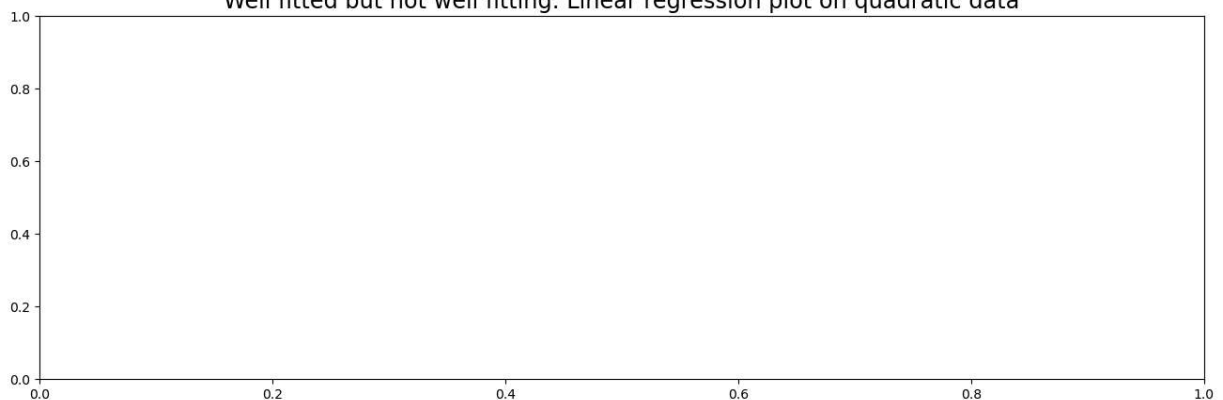
Linear Regression can be performed through Ordinary Least Squares (OLS) or Maximum Likelihood Estimation (MLE).

```
In [31]: # Generate data
x = np.random.uniform(low=20, high=260, size=100)
y = 50000 + 2000*x - 4.5 * x**2 + np.random.normal(size=100, loc=0, scale=10000)

# Plot data with Linear Regression
plt.figure(figsize=(16,5))
plt.title('Well fitted but not well fitting: Linear regression plot on quadratic da
sns.regplot(x, y)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[31], line 8
      6 plt.figure(figsize=(16,5))
      7 plt.title('Well fitted but not well fitting: Linear regression plot on quadr
atic data', fontsize='xx-large')
----> 8 sns.regplot(x, y)

TypeError: regplot() takes from 0 to 1 positional arguments but 2 were given
Well fitted but not well fitting: Linear regression plot on quadratic data
```



```
In [32]: # Linear regression from scratch
import random
# Create data from regression
xs = np.array(range(1,20))
```

```

ys = [0,8,10,8,15,20,26,29,38,35,40,60,50,61,70,75,80,88,96]

# Put data in dictionary
data = dict()
for i in list(xs):
    data.update({xs[i-1] : ys[i-1]})

# Slope
m = 0
# y intercept
b = 0
# Learning rate
lr = 0.0001
# Number of epochs
epochs = 100000

# Formula for Linear Line
def lin(x):
    return m * x + b

# Linear regression algorithm
for i in range(epochs):
    # Pick a random point and calculate vertical distance and horizontal distance
    rand_point = random.choice(list(data.items()))
    vert_dist = abs((m * rand_point[0] + b) - rand_point[1])
    hor_dist = rand_point[0]

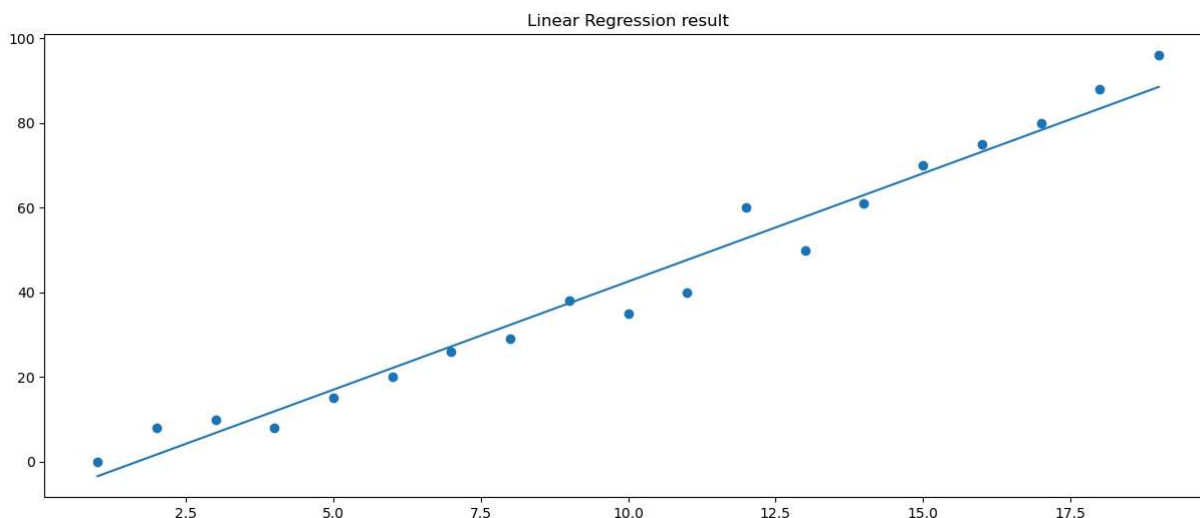
    if (m * rand_point[0] + b) - rand_point[1] < 0:
        # Adjust line upwards
        m += lr * vert_dist * hor_dist
        b += lr * vert_dist
    else:
        # Adjust line downwards
        m -= lr * vert_dist * hor_dist
        b -= lr * vert_dist

# Plot data points and regression line
plt.figure(figsize=(15,6))
plt.scatter(data.keys(), data.values())
plt.plot(xs, lin(xs))
plt.title('Linear Regression result')
print('Slope: {}\nIntercept: {}'.format(m, b))

```

Slope: 5.109811956273668

Intercept: -8.54138956559179



Bootstrapping

Bootstrapping is a resampling technique to quantify the uncertainty of an estimator given sample data. In other words, we have a sample of data and we take multiple samples from that sample. For example, with bootstrapping we can take means for each bootstrap sample and thereby make a distribution of means.

Once we created a distribution of estimators we can use this to make decisions.

Bootstrapping can be:

1. Non-parametric (Take random samples from sample)
2. Parametric (Use the sample to create new datasets with a (normal) distribution which has sample mean and variance). Downside: You are making assumptions about the distribution. Upside: Computationally more light
3. Online bootstrap (Take samples from a stream of data)

```
In [33]: # scikit-learn bootstrap package
from sklearn.utils import resample

# data sample
data = df['Income'].sample(10000)

# prepare bootstrap samples
boot = resample(data, replace=True, n_samples=10, random_state=1)
print('Means of Bootstrap Samples: \n{}\n'.format(boot))
print('Mean of the population: ', data.mean())
print('Standard Deviation of the population: ', data.std())

# Bootstrap plot
pd.plotting.bootstrap_plot(data)
```

Means of Bootstrap Samples:

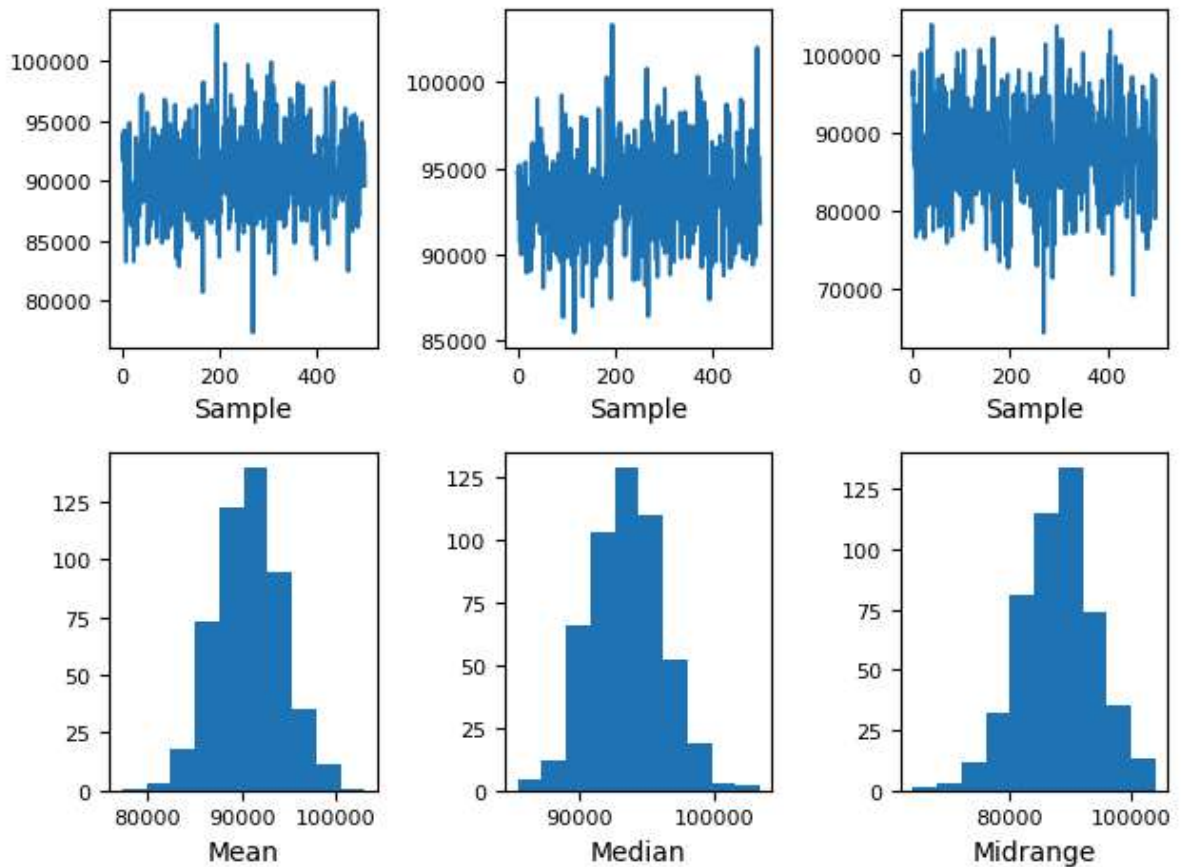
36892	88852.0
96741	99557.0
84743	81520.0
135716	108345.0
8652	41561.0
100350	91296.0
85693	99154.0
130160	63020.0
124192	105299.0
149657	89968.0

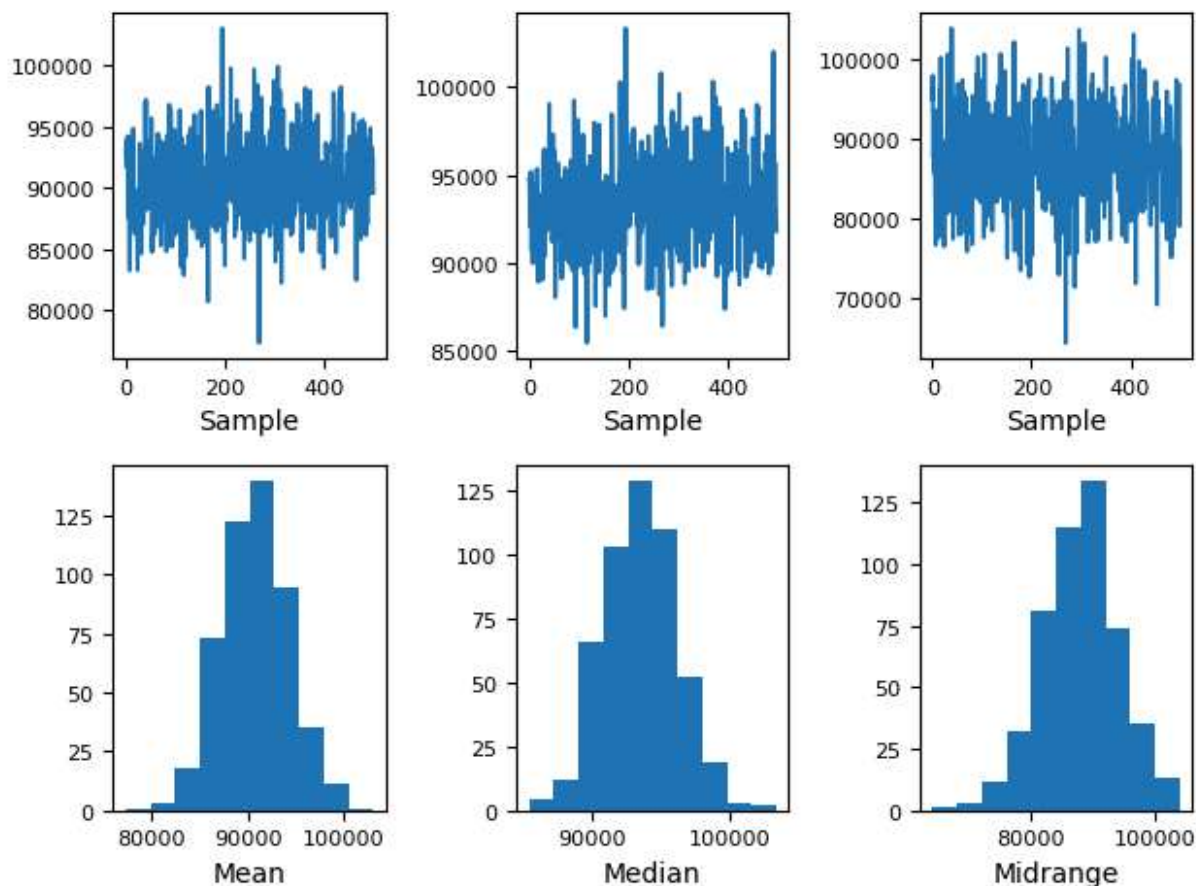
Name: Income, dtype: float64

Mean of the population: 90591.2046

Standard Deviation of the population: 24971.58823961066

Out[33]:





Hypothesis testing

P-Value

A p-value is the probability of finding equal or more extreme results when the null hypothesis (H_0) is true. In other words, a low p-value means that we have compelling evidence to reject the null hypothesis.

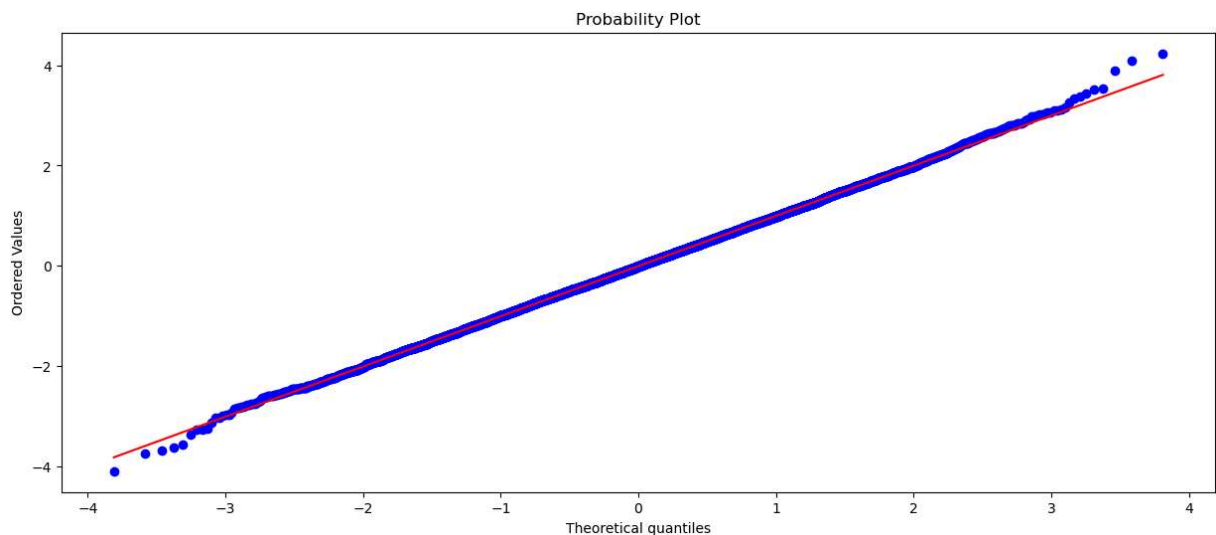
If the p-value is lower than 5% ($p < 0.05$). We often reject H_0 and accept H_a is true. We say that $p < 0.05$ is statistically significant, because there is less than 5% chance that we are wrong in rejecting the null hypothesis.

```
In [34]: # Perform t-test and compute p value of two random samples
print('T-statistics and p-values of two random samples.')
for _ in range(10):
    rand_sample1 = np.random.random_sample(10)
    rand_sample2 = np.random.random_sample(10)
    print(stats.ttest_ind(rand_sample1, rand_sample2))
```

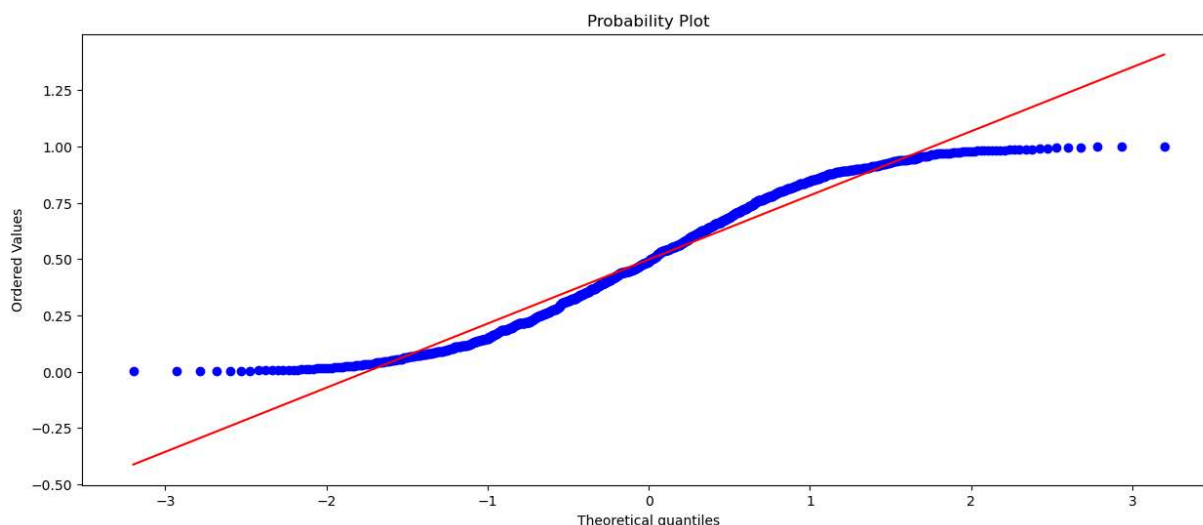
T-statistics and p-values of two random samples.

```
TtestResult(statistic=np.float64(-0.25524879241072257), pvalue=np.float64(0.80142469
78126562), df=np.float64(18.0))
TtestResult(statistic=np.float64(1.7424468410230753), pvalue=np.float64(0.0984895626
5325397), df=np.float64(18.0))
TtestResult(statistic=np.float64(0.4574072762415252), pvalue=np.float64(0.6528510140
272723), df=np.float64(18.0))
TtestResult(statistic=np.float64(-0.9864746803579468), pvalue=np.float64(0.336975798
8771255), df=np.float64(18.0))
TtestResult(statistic=np.float64(1.59557279543398), pvalue=np.float64(0.127991356095
3603), df=np.float64(18.0))
TtestResult(statistic=np.float64(1.844982015079353), pvalue=np.float64(0.08155916880
01063), df=np.float64(18.0))
TtestResult(statistic=np.float64(0.180960856844929), pvalue=np.float64(0.85842027451
00713), df=np.float64(18.0))
TtestResult(statistic=np.float64(-0.8214793673321894), pvalue=np.float64(0.422121089
58270456), df=np.float64(18.0))
TtestResult(statistic=np.float64(0.42779690252353925), pvalue=np.float64(0.673873461
5303232), df=np.float64(18.0))
TtestResult(statistic=np.float64(-1.3251733538325092), pvalue=np.float64(0.201689923
39706765), df=np.float64(18.0))
```

```
In [35]: # q-q plot of a normal distribution
plt.figure(figsize=(15,6))
stats.probplot(normal_dist, dist="norm", plot=plt)
plt.show()
```



```
In [36]: # q-q plot of a uniform/random distribution
plt.figure(figsize=(15,6))
stats.probplot(uniform_dist, dist="norm", plot=plt)
plt.show()
```



Outliers

An outlier is an observation which deviates from other observations. An outlier often stands out and could be an error.

Outliers can mess up your statistical models. However, outliers should only be removed when you have established good reasons for removing the outlier.

Sometimes the outliers are the main topic of interest. This is for example the case with fraud detection. There are many outlier detection methods, but here we will discuss Grubbs test and Tukey's method. Both tests assume that the data is normally distributed.

```
In [37]: # Detect outliers on the 'Income' column of the Toy Dataset

# Function for detecting outliers a la Tukey's method using z-scores
def tukey_outliers(data) -> list:
    # For more information on calculating the threshold check out:
    # https://medium.com/datadriveninvestor/finding-outliers-in-dataset-using-pytho
    threshold = 3

    mean = np.mean(data)
    std = np.std(data)

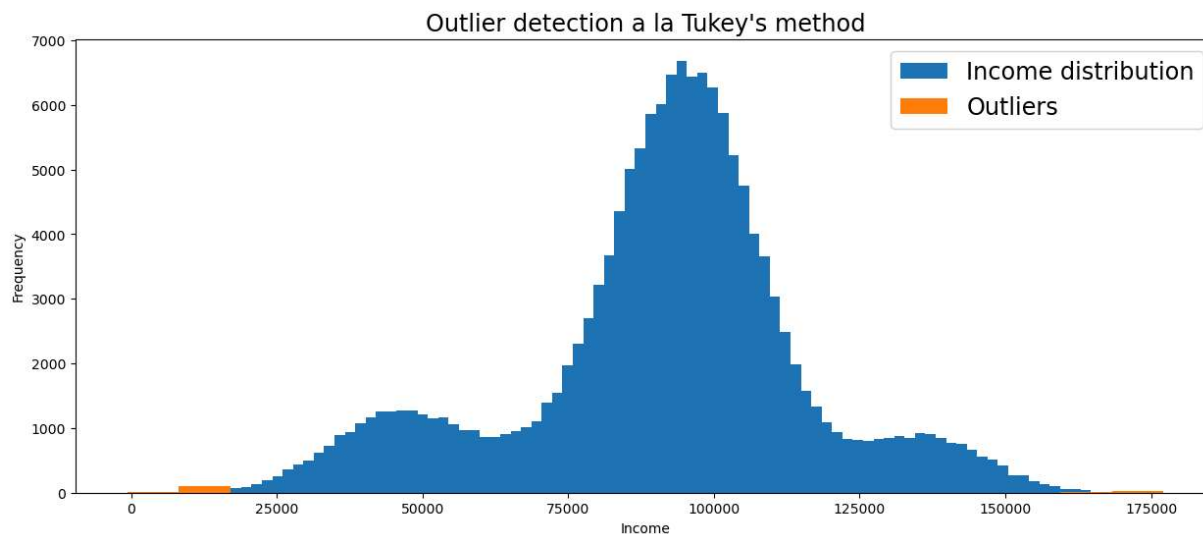
    # Spot and collect outliers
    outliers = []
    for i in data:
        z_score = (i - mean) / std
        if abs(z_score) > threshold:
            outliers.append(i)
    return outliers

# Get outliers
income_outliers = tukey_outliers(df['Income'])

# Visualize distribution and outliers
```

```
plt.figure(figsize=(15,6))
df['Income'].plot(kind='hist', bins=100, label='Income distribution')
plt.hist(income_outliers, bins=20, label='Outliers')
plt.title("Outlier detection a la Tukey's method", fontsize='xx-large')
plt.xlabel('Income')
plt.legend(fontsize='xx-large')
```

Out[37]: <matplotlib.legend.Legend at 0x1f91e861810>



Overfitting

Our model is overfitting if it is also modeling the 'noise' in the data. This implies that the model will not generalize well to new data even though the error on the training data becomes very small. Linear models are unlikely to overfit, but as models become more flexible we have to be wary of overfitting. Our model can also underfit which means that it has a large error on the training data.

Prevention of Overfitting

1. Split data into training data and test data.
2. Regularization: limit the flexibility of the model.
3. Cross Validation

Link functions

A Link Function is used in Generalized Linear Models (GLMs) to apply linear models for a continuous response variable given continuous and/or categorical predictors. A link function that is often used is called the inverse logit or logistic sigmoid function.

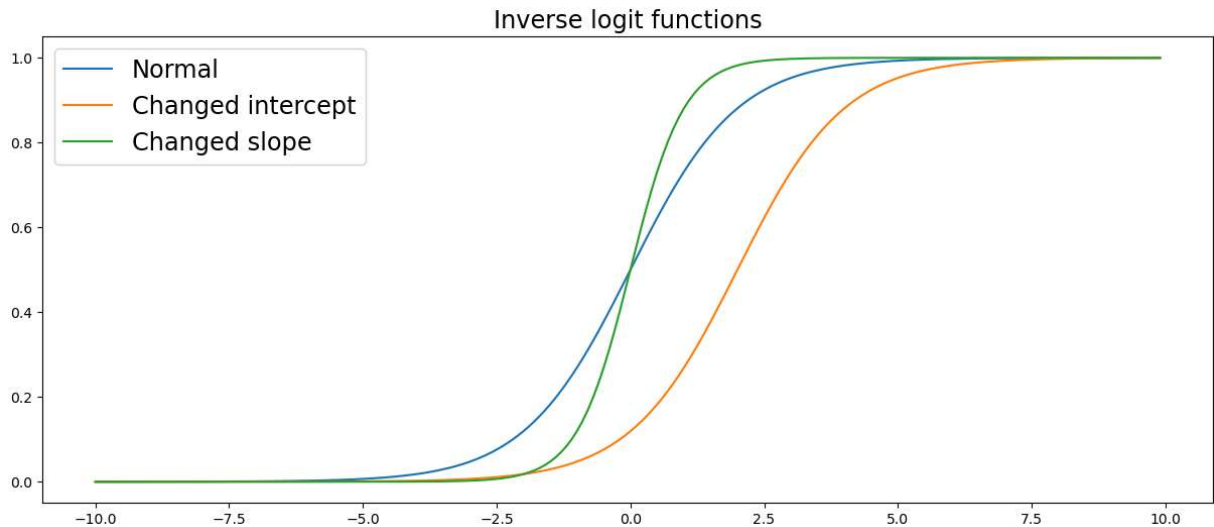
```
In [38]: # Inverse Logit function (link function)
def inv_logit(x):
    return 1 / (1 + np.exp(-x))
```

```

t1 = np.arange(-10, 10, 0.1)
plt.figure(figsize=(15,6))
plt.plot(t1, inv_logit(t1),
         t1, inv_logit(t1-2),
         t1, inv_logit(t1*2))
plt.title('Inverse logit functions', fontsize='xx-large')
plt.legend(('Normal', 'Changed intercept', 'Changed slope'), fontsize='xx-large')

```

Out[38]: <matplotlib.legend.Legend at 0x1f91ea520d0>



Logistic regression

With logistic regression we use a link function like the inverse logit function mentioned above to model a binary dependent variable. While a linear regression model predicts the expected value of y given x directly, a GLM uses a link function.

```

In [39]: # Simple example of Logistic Regression in Python
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)

# Logistic regression classifier
clf = LogisticRegression(random_state=0,
                        solver='lbfgs',
                        multi_class='multinomial').fit(X, y)

print('Accuracy score of logistic regression model on the Iris flower dataset: {}'.format(

```

Accuracy score of logistic regression model on the Iris flower dataset: 0.9733333333333334

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.7. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.  
  warnings.warn(  
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):  
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.  
  
Increase the number of iterations (max_iter) or scale the data as shown in:  
  https://scikit-learn.org/stable/modules/preprocessing.html  
Please also refer to the documentation for alternative solver options:  
  https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression  
n_iter_i = _check_optimize_result(
```

In []: